MECH321
Dynamics of Engineering Systems

Lecture 10 · Optional
PID Control

Prof. Zengxi (Stephen) Pan

TYUT · 2026

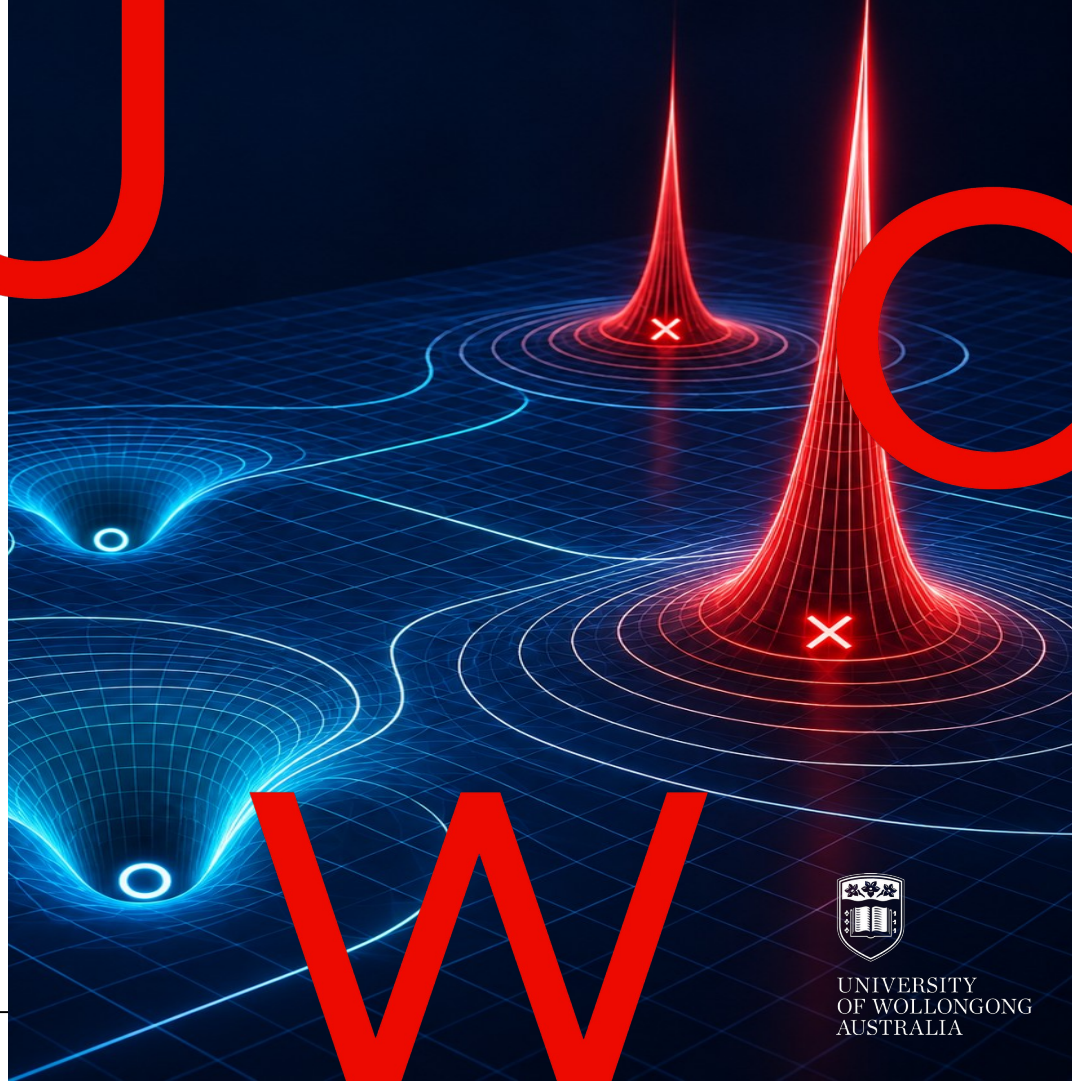


Table of Contents

PART 1 **PID actions and controller choice**

PART 2 **Controller design and implementation**

PART 3 **Practice and implementation extensions**

After Lecture 8's stability checks, compare P, PI and PID under actuator and sensor limits to make a feasible final design choice.

PART 1

PID actions and controller choice

Compare controller actions and calculate a complete proportional response.

01 P I and D actions

02 Controller comparison

03 Concept check

Before the lecture

OPTIONAL

Three questions from Home Reading 10

A Controller actions

What does each of the proportional, integral and derivative actions respond to, and how does each change the response?

B Integral trade-off

Why can integral action remove step error yet change the closed-loop poles and reduce stability margin?

C Physical limits

How can actuator saturation and integrator windup defeat an otherwise stable linear controller design?

Answer all three without notes

These are the questions the Home Reading was designed to prepare you for. Mark any answer you cannot yet explain.

The parallel PID law

OPTIONAL

Controller gains—not Lecture 7 tracking-error constants

$$u(t) = K_p e(t) + K_i \int e(t)\,dt + K_d \frac{de(t)}{dt}$$

$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

Proportional

the present

Reacts to the error that exists now

Bigger error, bigger correction

Integral

the past

Accumulates error over time

Keeps pushing until the error is gone

Derivative

the future

Reacts to how fast the error is changing

Brakes before the target is reached

State the form before you quote a gain

Many texts write $K_p\left[1 + \frac{1}{T_i s} + T_d s\right]$ instead. The same controller has completely different numbers in the two conventions, so a gain without its form is not information.

Proportional action

OPTIONAL

Effects of P gain depend on the plant and the chosen output

What raising K_p does

For stable Type 0 feedback, it reduces finite step error

For $G(s) = 2$ /(s+2), the pole is −(2+2K_p)

Positive K_p speeds this first-order response without overshoot

Extra poles or delay can instead limit the stable gain

Why the error never reaches zero

With $C = K_p$, the error is $\frac{1}{1+K_pG(0)}$

Zero error would need infinite gain

And a finite error is required to produce any output at all — that is the mechanism, not a defect

Lecture 9's s damping trade-off is model-specific

For its position plant 1/[s(s+2)], more positive gain reduces damping and raises overshoot, but the ideal closed loop stays stable. An eventual instability needs a different plant, extra dynamics or delay.

P control: calculate the entire step

EXAMPLE

A single pole still leaves a finite offset for this type-0 plant

Problem and closed loop

Take $G(s) = \frac{2}{s+2}$, unity feedback and $C(s) = K$. From $1 + KG$, the closed-loop transfer is $\frac{2K}{s+2+2K}$. At $K = 4$ its pole is -10 rad/s, so the time constant is 0.10 s. The DC gain is $8/10 = 0.8$.

Time response and physical meaning

For a unit step, $y(t) = 0.8\left(1 - e^{-10t}\right)$. Thus $y(0) = 0$, $y(\infty) = 0.8$ and the final error is 0.2. The controller needs a nonzero error to provide the steady command: $u(\infty) = 4 \times 0.2 = 0.8$. Raising K reduces the offset but also raises the initial request $u(0^+) = K$.

CHECK The final-value calculation and the explicit time response must agree.

Integral action

OPTIONAL

What it buys

An uncancelled integrator raises the loop type by one

Zero step error requires a stable loop and a feasible command

Integral output can remain nonzero when error is zero

Constant-load rejection depends on its entry path

What it costs

A pure integrator contributes −90° of phase

A PI controller has phase between −90° and 0°

Aggressive K_i gives slow oscillation, then instability

Saturation can cause windup; see slides 19–21

The trade, using Lecture 8 stability margins

Check phase margin at the new crossover. Integral action can remove steady error only when the loop remains stable and the actuator can supply the required output.

Derivative action

OPTIONAL

What it buys

Adds phase lead, up to $+90°$

Which is exactly what integral action took away

Improves damping, so overshoot falls

Allows a higher crossover frequency, hence a faster loop

What it costs

An ideal differentiator has gain proportional to ω

So it amplifies high-frequency sensor noise without limit

A noisy measurement becomes a violently chattering actuator

It is never implemented in its ideal form

Which is why the practical form is filtered

$\frac{K_d N s}{s+N}$. It behaves like a differentiator up to N rad/s and flattens out above it, so the high-frequency gain is capped at $K_d N$.

P, PI, PD and PID compared

OPTIONAL

Typical trends; verify the actual plant and tuned loop

Structure	Step error	Damping	Noise	Use when
P	finite (type 0)	depends on plant and K_p	low	offset is acceptable
PI	zero if stable	may reduce damping vs P	low	accuracy matters, plant is slow
PD	finite (type 0)	can improve	high	speed matters; offset acceptable
PID	zero if stable	can improve	high	accuracy and damping; clean sensor

Choose the simplest controller that meets the requirements

Zero offset requires stable integral action and feasible actuator demand. Derivative action needs filtering. Check poles, zeros and margins before claiming better damping.

Concept check

EXAMPLE

A symptom report from the plant floor. What do you change?

"With proportional control the loop is fast, but it settles about 4 % below the setpoint and stays there."
"We added integral action. The offset is gone, but now it oscillates slowly and takes far longer to settle."

A
Remove the integral term again

B
Increase K_p to speed it back up

C
Keep integral, change something else

Diagnose before you tune
Which requirement did each symptom come from, and which term addresses it?

Concept check — explained

EXAMPLE

C. Keep what solved the stated problem

Why not A

The offset was the reported fault

Removing integral action brings it straight back

That is undoing the fix, not improving it

Why not blindly choose B

Oscillation may come from low damping or saturation

Raising K_p has a plant-dependent effect

Predict its effect before changing the gain

What to do

Test lower K_i, and check whether filtered derivative helps

Then recompute the gain and phase margins

Verify with a step response, not by feel

The principle behind the answer

Preserve the accuracy requirement. Check the plant model, stability margins and actuator limits. Change one justified setting, then verify the response.

PART 2

Controller design and implementation

Match coefficients, verify the actual response, then check actuator limits.

01 PI design and actual step response

02 PID matching and model limits

03 Saturation and anti-windup

PI design: match two coefficients

EXAMPLE

Derive the gains before trusting a simulation

Define plant, controller and target

Use $G(s) = \frac{2}{s+1}$ and $C(s) = K_p + K_i/s$ in unity feedback. The closed-loop characteristic polynomial is $s(s+1) + 2(K_p s + K_i)$, which expands to $s^2 + (1 + 2K_p)s + 2K_i$. Choose the repeated-pole target $(s+2)^2 = s^2 + 4s + 4$.

Solve and verify

Match the s coefficient: $1 + 2K_p = 4$, giving $K_p = 1.5$. Match the constant: $2K_i = 4$, giving $K_i = 2$. Substitute back to recover $s^2 + 4s + 4$. The integral term adds a loop pole at the origin, so a stable closed loop has zero final error to a unit step.

CHECK Pole matching determines the denominator, not the shape of the whole response.

PI design: check the numerator and peak

EXAMPLE

Repeated real poles do not guarantee zero overshoot

Construct the full transfer

With $K_p = 1.5$ and $K_i = 2$, the closed-loop numerator is $3s + 4$, so $T(s) = \frac{3s+4}{(s+2)^2}$. For a unit step, partial fractions give $y(t) = 1 - e^{-2t} + te^{-2t}$. The closed-loop zero changes the shape despite both poles being at −2.

Find the actual maximum

Differentiate: $y\prime(t) = (3 - 2t)e^{-2t}$. The output peaks at $t = 1.5$ s, with $y = 1 + 0.5e^{-3} \approx 1.0249$: about 2.49% overshoot. It first reaches the setpoint at $t = 1$ s, then briefly exceeds it. A pole-only claim of “no overshoot” would be wrong.

CHECK Always inspect zeros and the complete response after coefficient matching.

PI actuator test and final decision

OPTIONAL

A mathematically correct controller can fail an initial physical limit

Check startup and steady command

For the worked PI controller, a unit step initially has $e = 1$ and zero integral state, so $u\left(0^{+}\right) = K_p = 1.5$. At steady state the plant needs input 0.5 to hold output one because $G\left(0\right) = 2$. If the actuator limit is ±1, this design saturates initially even though its final command is feasible.

Independent controller-design extension

On this separate practice plant, simulate actuator limits with anti-windup and compare P, PI and optional filtered PID using the same plant and requirements. Recommend the simplest controller that meets all limits, or state why none does.

CHECK **The recommended controller must be feasible in both transient and steady operation.**

PID Coefficient Matching

EXAMPLE

The algebra gives a starting controller, not a released design

Match a third-order target

Use $G(s) = \frac{1}{s(s+2)}$ and ideal parallel $C(s) = K_d s + K_p + K_i/s$. The characteristic polynomial is $s^3 + (2 + K_d)s^2 + K_p s + K_i$. Match the target $(s+2)\left(s^2 + 4s + 8\right) = s^3 + 6s^2 + 16s + 16$; hence $K_d = 4$, $K_p = 16$ and $K_i = 16$.

Check what was not designed

The closed-loop transfer has numerator $4s^2 + 16s + 16$, so its zeros and step shape also matter. Ideal differentiation strongly amplifies sensor noise and a setpoint step can cause a derivative kick. Implement filtered derivative on measurement, include actuator limits, then recheck margins and time response.

CHECK State whether gains use the parallel, ideal or series PID convention.

Why that method stops working

OPTIONAL

Pole placement by coefficient matching has a limit

When it works

The number of free gains equals the number of coefficients to be matched

A first-order plant with PI: two gains, two coefficients

A second-order plant with PID: three gains, three coefficients

When it does not

A higher-order plant has more coefficients than a PID has gains

You cannot place every pole where you want it

Then the design becomes loop shaping — choose the frequency response and accept whatever poles follow

Why the Lecture 8 margin check matters

For most plants you do not place poles — you shape the loop and verify with margins. Pole placement is the special case where the arithmetic happens to close.

Saturation and integral windup

OPTIONAL

The failure that no linear analysis predicts

1. **A large command arrives**
   The error is big, so the controller demands a large u
2. **The actuator hits its limit**
   Output is limited; feedback remains connected
3. **The integrator keeps integrating**
   The error is still non-zero, so $K_i \int e(t)\,dt$ grows and grows
4. **The output finally reaches the setpoint**
   But the integrator holds a huge stored value
5. **The loop overshoots badly**
   It must now accumulate negative error just to unwind

The signature on a plot

Slow recovery and large overshoot can reflect windup, beyond the linear prediction using ζ. Check clipping and the integral state before retuning.

Windup: separate error from clipping

EXAMPLE

The integrator accumulates tracking error, not the actuator command deficit

A correct numerical calculation

Suppose $K_i = 2$ control units/(error unit·s), and error stays at +1 for a 2 s saturation interval. The integral contribution grows by K_i times the error area: $2 \times (1 \times 2) = 4$ control units. If the requested command is 12 while the actuator can deliver 5, the seven-unit clipping gap is a different quantity.

Why anti-windup needs both signals

The persistent positive error can leave the integral state high even after output reaches the setpoint. Back-calculation uses the command gap (actual minus requested) to push that state down; conditional integration freezes it only while error would drive farther into saturation. Both require the physical limit in the model.

CHECK Plot requested u, delivered u, error and integral state separately.

Anti-windup

OPTIONAL

Two standard fixes, both simple

Conditional integration

Freeze if error deepens saturation

Resume when error drives back toward range

One line of code, and it removes most of the problem

Back-calculation

Feed the difference between the commanded u and the delivered u back into the integrator

It unwinds smoothly rather than freezing

The standard method in industrial controllers

And the check that must be run

Any simulation used for final tuning has to include the actuator limit. A linear simulation of a saturating plant is not evidence about the machine you are building.

Six checks before implementation

OPTIONAL

Extended checks supporting Tutorial 10 Q3 and final design

1. **Transient response**
   Overshoot and settling time against the specification
2. **Stability margins**
   Gain and phase margin, with crossover frequencies
3. **Noise and sensitivity**
   Peak of $|S|$, and what the actuator does with sensor noise
4. **Parameter variation**
   Recompute everything at the worst credible parameters
5. **Neglected dynamics**
   Resonances and delays outside the identified range
6. **Saturation and windup**
   Rate and amplitude limits, with anti-windup in place

Meeting the specification in the nominal linear model
is the first of six checks, not the conclusion. A controller is acceptable only when the complete picture holds together.

PART 3

Practice and implementation extensions

Optional self-study beyond the one-hour teaching route.

01 PI practice with worked answers

02 Classical tuning reference

03 Sampling and digital implementation

Mini-test

OPTIONAL

Twelve minutes, without notes

QUESTION 1

Design

$G = \frac{2}{s+4}$, unity feedback, $C = K_p + \frac{K_i}{s}$

Write the characteristic equation

Choose K_p and K_i to place both poles at -5

QUESTION 2

Judge

Which term removes steady-state error, and why?

Which term improves damping, and what does it cost?

Name two checks that a linear simulation cannot perform

Question 1 has repeated poles

Which means the target polynomial is $(s+5)^2$, not $(s+5)(s+4)$.

After-class slide — not covered in the contact hour.

Mini-test: solve and verify the PI gains

OPTIONAL

After-class coefficient matching for the stated repeated-pole target

Question 1: complete the algebra

With $G = \frac{2}{s+4}$ and $C = K_p + K_i/s$, multiply $1 + CG = 0$ by $s\,(s+4)$. The polynomial becomes $s^2 + (4 + 2K_p)s + 2K_i$. Desired poles at −5 give $(s+5)^2 = s^2 + 10s + 25$. Match coefficients: $4 + 2K_p = 10$ gives $K_p = 3$; $2K_i = 25$ gives $K_i = 12.5$. Substitution reproduces the exact target denominator.

Question 2: make a design judgement

Integral action raises loop type and removes a stable step offset. Derivative action can improve damping, but a real derivative requires filtering to avoid excessive sensor-noise amplification. A linear simulation alone cannot verify saturation/windup or a response outside the validated plant frequency range; those require explicit models and tests.

CHECK **Real poles specify the denominator; inspect the closed-loop numerator before claiming zero overshoot.**

Ziegler–Nichols in full

Both classical methods, with their caveats

Controller	K_p	T_i	Td
P	0.5 K_u	—	—
PI	0.45 K_u	$\frac{T_u}{1.2}$	—
PID	0.6 K_u	$\frac{T_u}{2}$	$\frac{T_u}{8}$

Three warnings

The ultimate-gain test deliberately drives a real plant to sustained oscillation, which is unacceptable on much equipment. The rules target disturbance rejection, so setpoint overshoot is often near 25 %. And they assume a plant shape that yours may not have.

After-class slide — not covered in the contact hour.

Digital Implementation

Because the controller will be code

Sampling adds delay

Roughly half a sample period, on average

It costs phase margin, at $57.3 \cdot \omega_{gc} \cdot \frac{T}{2}$ degrees

Sample at least 20 times the crossover frequency

The integral becomes a sum

$I \leftarrow I + K_i \cdot e \cdot T_s$

Which is where windup physically lives in the code

Clamp it there

The derivative becomes a difference

$\frac{e_k - e_{k-1}}{T_s}$ amplifies quantisation noise

Filter it, and take the derivative of the measurement, not of the error

Why derivative on measurement, not on error

A step change in the setpoint makes $\frac{de}{dt}$ momentarily enormous, and the actuator gets a violent kick. Differentiating $-y$ instead removes that without changing the response to a disturbance.

Discrete PID

Unfiltered schematic; not a complete implementation

Position form, executed once per sample period T_s:

$e = r - y$;

I = I + Ki * e * Ts; % bare integral update; no anti-windup shown

D = −Kd * (y − y_{prev}) / Ts; % unfiltered derivative on measurement

u = Kp * e + I + D;

u = max(umin, min(umax, u)); % output limit alone does not prevent windup

Implementation steps still required

Add derivative filtering, update the previous measurement each sample, and implement conditional integration or back-calculation. Set initial states and test actuator limits before use.

After-class slide — not covered in the contact hour.

Velocity Form

Incremental output stores the controller history

The idea

Compute the CHANGE in u each sample, not u itself

$$\Delta u = K_p \Delta e + K_i e T_s + \frac{K_d \left(\Delta^2 e\right)}{T_s}$$

Then $u \leftarrow u + \Delta u$

Implementation cautions

Accumulated output can still wind up during saturation

Bumpless transfer needs output tracking and initialization

It directly supports actuators with incremental commands

Same controller, different arithmetic

Position and velocity forms are equivalent with consistent discretization and initialization. Saturation handling, derivative filtering and state tracking still require explicit design.

After-class slide — not covered in the contact hour.

Tutorial and self-study

OPTIONAL

Practise controller reasoning after the lecture

OPTIONAL PRACTICE

Tutorial 10

Q1 basic checks and conditional integration.
Q2 thermal P/PI and heater feasibility.
Q3 PID audit; Q4 filtered derivative after class.

REVIEW

Home Reading 10

The parallel PID law and its conventions
Proportional and integral action, and their costs
Derivative action and filtering
Saturation, windup and the integrated design workflow

Where this goes after the subject

Root-locus and frequency-response controller design, state feedback and observers, digital control, and system identification from data — all of which start from exactly what you built this week.

Reference: Ogata, System Dynamics 4th ed. §§10-3, 10-6 and 10-10; Modern Control Engineering 5th ed. §§8-1 to 8-5.


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!